Jobs, Stages, and Tasks in Apache Spark

In Apache Spark, every application is executed in **three levels**:

1. **Job** – The complete unit of work triggered by an action.
2. **Stage** – A group of tasks that can run without a shuffle.
3. **Task** – The smallest unit of execution that processes one partition.

The execution hierarchy is:

```
Spark Application
       |
       ▼
      Job
       |
       ▼
    Stage(s)
       |
       ▼
    Task(s)
```

---

1. Job

What is a Job?

A **Job** is the **highest-level unit of execution** in Spark.

A new job is created **every time an action is called**.

Examples of actions that create jobs:

- `show()`
- `count()`
- `collect()`
- `take()`
- `first()`
- `write.csv()`
- `write.parquet()`

Example

```
df = spark.read.csv("employees.csv", header=True)

df.filter(df.salary > 5000).show()
```

Execution:

```
filter()   → Transformation
show()     → Action

Result:
1 Spark Job
```

If you execute:

```
df.count()

df.show()
```

Spark creates:

```
count() → Job 1

show()  → Job 2
```

Each action starts a new job.

---

2. Stage

What is a Stage?

A **Stage** is a **group of tasks that can execute without requiring a shuffle**.

Spark divides a job into multiple stages whenever it encounters a **wide transformation**.

A new stage is created after operations such as:

- `groupBy()`
- `join()`
- `distinct()`
- `orderBy()`
- `repartition()`

These operations require a **shuffle**, which creates a stage boundary.

Example

```
df.filter(df.salary > 5000) \
  .groupBy("department") \
  .count() \
  .show()
```

Execution:

```
Stage 1
--------
Read CSV
Filter

      Shuffle

Stage 2
--------
GroupBy
Count
Show
```

Here:

- **Stage 1** performs the read and filter.
- The shuffle occurs.
- **Stage 2** performs the aggregation and finishes the action.

Narrow vs Wide Transformations

Narrow Transformation

No shuffle is required.

Examples:

- `filter()`
- `select()`
- `withColumn()`
- `map()`

These remain in the same stage.

Wide Transformation

Requires a shuffle.

Examples:

- `groupBy()`

- `join()`
- `distinct()`
- `orderBy()`
- `repartition()`

Each shuffle creates a new stage.

---

3. Task

What is a Task?

A **Task** is the **smallest unit of execution** in Spark.

Each task processes **one data partition**.

Example

Suppose your DataFrame has **8 partitions**.

```
Dataset
     │
     ▼
8 Partitions
```

Spark creates:

```
8 Tasks
```

Each task processes one partition.

If a stage has **200 partitions**, Spark creates **200 tasks**.

Complete Execution Flow

Suppose we execute:

```
df = spark.read.csv("employees.csv", header=True)

result = (
    df.filter(df.salary > 5000)
      .groupBy("department")
      .count()
```

```
)

result.show()
```

Execution flow:

```
Action (show)
      │
      ▼
     Job
      │
      ▼
────────────────────────────
Stage 1
────────────────────────────
Read CSV
Filter

      Shuffle

────────────────────────────
Stage 2
────────────────────────────
GroupBy
Count
Show

Each Stage
      │
      ▼
Tasks
(one task per partition)
```

Relationship Between Job, Stage, and Task

```
Spark Application
       │
       ▼
Job
 │
 ├── Stage 1
 │     ├── Task 1
 │     ├── Task 2
 │     ├── Task 3
 │     └── Task 4
 │
 └── Stage 2
       ├── Task 1
       ├── Task 2
       ├── Task 3
       └── Task 4
```

Real-World Example

Imagine a company processes **500 GB** of sales data.

The data is divided into **100 partitions**.

The query is:

```
sales.filter(...)
     .join(customers, ...)
     .groupBy("country")
     .sum("sales")
     .show()
```

Spark executes:

- **1 Job** (because of `show()`).
- **Multiple Stages** (because `join()` and `groupBy()` require shuffles).
- **100 Tasks per Stage** (assuming 100 partitions).

Each executor processes tasks in parallel.

Comparison Table

Feature	Job	Stage	Task
Definition	Complete unit of work triggered by an action	Group of operations executed without a shuffle	Smallest unit of execution
Created By	Action (`show()`, `count()`, `collect()`)	Shuffle boundaries (wide transformations)	Data partitions
Execution Level	Highest	Middle	Lowest
Contains	Stages	Tasks	Processes one partition
Runs On	Driver coordinates the job	Executors execute stages through tasks	Executors execute tasks

---

Interview Questions

Q1. What is a Job in Spark?

A **Job** is the highest-level unit of execution. It is created whenever an **action** is called and represents the complete work required to produce the requested result.

Q2. What is a Stage?

A **Stage** is a group of tasks that can execute without a shuffle. Spark creates a new stage whenever it encounters a **wide transformation** that requires data shuffling.

Q3. What is a Task?

A **Task** is the smallest unit of execution in Spark. Each task processes **one data partition** and runs on an executor.

Q4. What creates a new Job?

An **action** such as:

- `show()`
- `count()`
- `collect()`
- `write.parquet()`

creates a new Spark job.

Q5. What creates a new Stage?

A **shuffle**, usually caused by wide transformations such as:

- `groupBy()`
- `join()`
- `distinct()`
- `orderBy()`
- `repartition()`

creates a new stage.

Q6. What determines the number of Tasks?

The **number of partitions** in a stage determines the number of tasks.

For example:

- 100 partitions → 100 tasks
- 500 partitions → 500 tasks

Interview Summary

- A **Job** is the highest-level execution unit in Spark and is created whenever an **action** is called.
- A **Stage** is a group of operations that can execute without a shuffle. Spark creates new stages at **shuffle boundaries**, typically after wide transformations.
- A **Task** is the smallest unit of execution. Each task processes **one partition** of data and runs on an executor.
- The execution hierarchy in Spark is:

```
Spark Application
      ↓
     Job
      ↓
   Stage(s)
      ↓
   Task(s)
```

- Remember these key rules for interviews:
  - **Action → creates a Job**
  - **Shuffle (wide transformation) → creates a new Stage**
  - **Partition → creates a Task**